Review: Basic OOP (Popquiz)

1. What is the main focus of procedural programming?

   Creating reusable functions or procedures to structure programs.

2. What is the main focus of object-oriented programming (OOP)?

   Modeling systems as interacting objects with their own state and behavior.

3. What are three key principles of OOP? Can you think of an example?

   1. Encapsulation (e.g. `private` data members)
   2. Inheritance (e.g. `housefly` / `mosquito` classes as `Insects`)
   3. Polymorphism (e.g. overloading an operator like `+`)

4. In the light-switch example, how does the `Switch` communicate with the `Light`?

   By sending messages (calling member functions) such as `Light::turnOn()` and `Light::turnOff()` to a `Light` object like `lamp`.

5. What is **encapsulation** in OOP?

   Hiding internal data and implementation details from users of a class.

6. What is a **class** in C++?

   A user-defined type that serves as a blueprint for creating objects.

7. What is an **object** in C++?

   An instance of a class containing data (state) and methods (behavior).

8. How does the car example illustrate interface vs. implementation?

   The driver interacts with the **interface** (wheel, pedals) only, without accessing internal mechanisms (**implementation**).

9. Why might OOP be unnecessary for simple programs?

   Because the added structure and class hierarchy can create overhead and complexity. For example in the `add` -> `class Adder` example.

10. Is this definition allowed? Will it compile?

```
class foo {
  // nothing to see here
};
```

11. What does the `const` keywoard after the function name mean?

```
double getBalance() const
{ return balance; }
```

12. If `Car` is a `class` with a member function `Car(int cylinders)`, how could you construct a `Car` object `chevy` with 4 cylinders in `main`? Write the statement.

```
Car(int cyl) : cylinders(cyl) {}
```

13. You cannot get the `Car` object data given the following definition

```
class Car {
private:
  int cylinders;
public:
};
```

Write a new member function `getCylinders` for the class that allows you to retrieve the number of cylinders.

```
int getCylinders() const { return cylinders; } // <-- get cylinder data
```

14. Write a statement to print the number of cylinders of a `Car` object named `chevy` with 4 cylinders.

```
Car chevy(4); // creates a 4-cylinder Car called 'chevy'.
cout << "Car has" << chevy.getCylinders() << " cylinders\n"; // <-- print
```

15. How many `Car` objects can you create in the same `main` program?

As many as you like (as long as there's memory available):

```
class Car{
private:
  int cylinders;
public:
  Car(int cyl) : cylinders(cyl) {} // constructor
  int getCylinders() const { return cylinders; } // get cylinder data
};

int main()
{
  Car chevy(4); // <-- creates a 4-cylinder Car called 'chevy'.
  cout << "This car has " << chevy.getCylinders() << " cylinders\n";
  Car ford(8); // <-- creates an 8-cylinder Car called 'ford'.
  cout << "This car has " << ford.getCylinders() << " cylinders\n";

  return 0;
}
```

```
This car has 4 cylinders
This car has 8 cylinders
```